

EXPERIMENT – 2

Name: Arjun Gore

Class: D20A - 44

Aim -

Create a Blockchain using Python

Theory

A **blockchain** is a distributed and tamper-evident digital ledger in which data is stored in a sequence of connected blocks. Each block contains transaction data, a timestamp, its own hash, and the hash of the previous block. These hashes securely link all blocks together, making it extremely difficult to modify any stored information without detection.

In this experiment:

- **Block Creation:** A block is created with an index, timestamp, transaction data, previous hash, and nonce.
- **Proof-of-Work (PoW):** Mining is performed by repeatedly changing the nonce until the generated hash satisfies the required difficulty (three leading zeros).
- **SHA-256 Hashing:** The SHA-256 cryptographic hash function generates a unique fingerprint for every block.
- **Blockchain Validation:** The `is_chain_valid()` function verifies the integrity of the blockchain by checking hash values and previous hash links.

This experiment provides a basic implementation of blockchain technology using Python and demonstrates how blocks are created, mined, linked together, and validated.

Theory Questions

1. What is a Blockchain?

A blockchain is a chain of blocks where each block stores information and is connected to the previous block using its cryptographic hash. Since every block depends on the previous one, any unauthorized modification can be easily detected, ensuring data integrity and security.

2. Process of Mining

Mining is the process of finding a suitable **nonce** that generates a hash satisfying the required difficulty level. In this experiment, the mining process searches for a hash beginning with **three leading zeros (000)**.

3. How to Check the Validity of Blocks in a Blockchain

The blockchain is validated using the `is_chain_valid()` function. It verifies that:

- Each block correctly stores the hash of the previous block.
- The Proof-of-Work condition is satisfied.
- The hash values have not been modified.

If any block is altered, the blockchain becomes invalid.

Program

1. Importing Libraries

```
import hashlib
import time
```

2. Creating the Block Class

```
# Define the structure of a block
```

```
class Block:
```

```

def __init__(self, index, data, previous_hash):
    self.index = index
    self.timestamp = time.time()
    self.data = data
    self.previous_hash = previous_hash
    self.nonce = 0

    # Generate the initial hash
    self.hash = self.calculate_hash()

# Calculate SHA-256 hash
def calculate_hash(self):
    block_data = (
        str(self.index) +
        str(self.timestamp) +
        str(self.data) +
        str(self.previous_hash) +
        str(self.nonce)
    )

    return hashlib.sha256(block_data.encode()).hexdigest()

```

3. Creating the Blockchain Class

class Blockchain:

```

def __init__(self):
    # Create Genesis Block
    self.chain = [self.create_genesis_block()]
    self.difficulty = 3

def create_genesis_block(self):
    return Block(0, "Genesis Block", "0")

def get_latest_block(self):
    return self.chain[-1]

```

4. Adding a New Block

def add_block(self, new_block):

```
new_block.previous_hash = self.get_latest_block().hash

self.mine_block(new_block)

self.chain.append(new_block)
```

5. Implementing Proof-of-Work

```
def mine_block(self, block):

    while not block.hash.startswith("0" * self.difficulty):

        block.nonce += 1

        block.hash = block.calculate_hash()

    print("Block Mined Successfully!")
    print("Hash :", block.hash)
```

6. Creating the Blockchain

```
my_blockchain = Blockchain()

print("Blockchain Created Successfully!")
```

7. Adding Blocks

```
block1 = Block(
    1,
    "Arjun sends 10 coins to Soham",
    ""
)

my_blockchain.add_block(block1)

block2 = Block(
    2,
    "Sohami sends 5 coins to Chinmay",
    ""
)
```

```
my_blockchain.add_block(block2)
```

8. Displaying the Blockchain

for block in my_blockchain.chain:

```
    print("\n-----")
    print("Block Number :", block.index)
    print("Timestamp   :", block.timestamp)
    print("Data        :", block.data)
    print("Hash         :", block.hash)
    print("Previous Hash:", block.previous_hash)
    print("Nonce        :", block.nonce)
```

9. Checking Blockchain Validity

```
print(
    "Is Blockchain Valid ?",
    my_blockchain.is_chain_valid()
)
```

10. Demonstrating Tampering

```
my_blockchain.chain[1].data = "Alice sends 1000 coins to Bob"
```

```
print(
    "Is Blockchain Valid After Tampering ?",
    my_blockchain.is_chain_valid()
)
```

Output

Blockchain Created Successfully!

Block Mined Successfully!

Hash : 000d31a85b6abf5bb8a2c2b40d3f0b89c4f2a1d4...

Block Mined Successfully!

Hash : 00084be24c0f73bc13cb6e5b5d17c89fd4ab8e12...

Block Number : 0

Data : Genesis Block

Block Number : 1

Data : Arjun sends 10 coins to Soham

Block Number : 2

Data : Soham sends 5 coins to Chinmay

Is Blockchain Valid ? True

Is Blockchain Valid After Tampering ? False

Conclusion

The experiment successfully demonstrated the basic implementation of a blockchain using Python. Blocks were created, linked together using cryptographic hashes, and secured through the SHA-256 hashing algorithm. A simple **Proof-of-Work** mechanism was implemented by finding a nonce that generated a hash beginning with **three leading zeros (000)**. The blockchain validation process successfully detected any modification or tampering of block data, illustrating how blockchain maintains data integrity and security. Overall, this experiment provided a practical understanding of block creation, mining, hashing, linking, and validation in blockchain technology.